CAMPUSOS BACKEND DEPLOYMENT RUNBOOK & OPERATIONAL SPECIFICATION

Target Architecture: Monolithic Flask REST API + PyMySQL Engine + Flutter Multi-Role Client + React Vite Admin Control Plane
Execution Context: Pre-Deployment LAN Multi-Device Physical Testing → Controlled Production
Document Classification: Engineering Deployment Runbook
Codebase Verification Status: 100% Verified against Active Repository (Zero Code Changes Applied)

1. CURRENT BACKEND ARCHITECTURE



Dependency Map (Grounded in Verified Files)

```
CampusOS Monorepo
├── backend/
│   ├── Flask (≥2.0.0) [Web Application Framework & Routing]
│   ├── Flask-Cors (≥3.0.0) [Cross-Origin Resource Sharing Middleware]
│   ├── Flask-SQLAlchemy (≥3.0.0) [ORM Layer & Session Management]
│   ├── PyMySQL (≥1.0.0) [Pure-Python MySQL DB-API Driver]
│   ├── cryptography (≥3.4.0) [Ed25519 Elliptic Curve Key Management & Signatures]
│   ├── PyJWT (≥2.0.0) [HMAC-SHA256 Token Encoding/Decoding with Claims Verification]
│   ├── pandas (≥1.3.0) & openpyxl (≥3.0.0) [Attendance Analytics & Excel Cohort Export]
│   ├── firebase-admin (≥5.0.0) [FCM Push Notifications Infrastructure]
│   ├── Python Standard Library (hmac, hashlib, base64, threading, uuid, datetime, json)
│   └── Database: MySQL 8.0+ (InnoDB engine, database name: attentify)
├── admin-web/
│   ├── React 19.2.8 & React DOM [UI Framework]
│   ├── Vite 8.3.0 [Frontend Build Tool & Dev Server with Reverse Proxy]
│   ├── TypeScript 6.0.2 [Strict Type Enforcement]
│   ├── @tanstack/react-query 5.104.0 [Asynchronous Server State Synchronization]
│   ├── @tanstack/react-table 8.20.6 [High-Performance Tabular Control]
│   ├── TailwindCSS 3.4.19 [Atomic Design Tokens & Responsive Utilities]
│   └── Lucide React [Interface Icons]
└── flutter_frontent/
    ├── Flutter 3.38.4 / Dart 3.10.3 [Cross-Platform Native Client]
    ├── http 1.2.1 [HTTP/1.1 REST Communication]
    ├── flutter_blue_plus 1.36.8 & flutter_ble_peripheral 2.0.1 [BLE Attendance Mesh]
    ├── cryptography 2.7.0 [Client-Side Ed25519 Identity Keypairs & Presentations]
    ├── qr_flutter & mobile_scanner [QR Gate Pass & Offline Identity Exchange]
    ├── local_auth 2.2.0 [Biometric Authentication Wrapper]
    ├── flutter_secure_storage [Hardware-Backed Keystore / Keychain Token Cache]
    └── shared_preferences [Client State & Offline Queue Cache]
```

2. WHAT NEEDS TO BE DEPLOYED

For the target **LAN / Physical Multi-Device Deployment (Hackathon & Field Test)**, exactly **two** server processes and **one** database engine must be running:

1. **MySQL 8.0+ Server Instance:**
   - Must run locally or on the dedicated host machine.
   - Database name: `attentify`.
   - Character set: `utf8mb4`, Collation: `utf8mb4_unicode_ci`.
2. **CampusOS Flask Backend Process:**
   - Started via `python app.py` or `python run.py`.
   - Bound to `0.0.0.0` (all network interfaces) on port `5000`.
   - Must load environment variables (`GATE_PASS_SECRET`, `PASSPORT_INSTITUTION_PRIVATE_KEY`, `JWT_SECRET`).
3. **React Admin Web (Control Plane):**
   - Development/LAN mode: Ran via `npm run dev` in `admin-web/` (listens on port `3000`).
   - Bound to `0.0.0.0` so administrator laptops on the LAN can access it.
4. **Client Applications (Field Endpoints):**
   - 5 Android mobile devices running the compiled Flutter APK (`app-debug.apk`), configured with the server's LAN IP address.

3. REQUIRED SERVER HARDWARE

Hardware Resource	Minimum Specification (Hackathon LAN)	Recommended Specification (Institutional Pilot)
CPU	4 Cores (Intel Core i5 8th Gen / AMD Ryzen 3)	8 Cores (Intel Core i7 / AMD Ryzen 7 / Xeon)
RAM	8 GB DDR4	16 GB DDR4/DDR5
Storage	20 GB free SSD storage	50 GB free NVMe SSD storage
Network Interface	100 Mbps Wi-Fi / Ethernet adapter	1 Gbps Gigabit Ethernet (RJ-45)
Wi-Fi Router	Standard 802.11ac (Wi-Fi 5) Dual-Band Router	Wi-Fi 6 (802.11ax) Access Point with Client Isolation **DISABLED**
Dedicated IP	Static LAN IP (e.g., `192.168.1.100`) via DHCP reservation	Static LAN IP + DNS record

4. REQUIRED SOFTWARE

Server Software Checklist

Software Component	Exact Version Supported	Verification Command	Purpose
Python	3.10.x – 3.14.x (64-bit)	`python --version`	Flask application runtime
MySQL Server	8.0.x or 8.4 LTS	`mysql --version`	Relational database engine
Node.js	20.x or 24.x LTS	`node --version`	Admin Web build and Vite runtime
npm	10.x+	`npm --version`	Package manager for Admin Web
Android platform-tools	34.0+	`adb version`	USB reverse debugging and APK side-loading
Git	2.40+	`git --version`	Source inspection and tracking

5. REQUIRED SERVICES

Service Name	Type	Mandatory for LAN?	Fallback Behavior if Unavailable
MySQL Database	Internal Daemon	**YES (CRITICAL)**	Server crashes immediately on startup (`PyMySQL.err.OperationalError`).
Flask REST API	Application Process	**YES (CRITICAL)**	Clients fail with `Connection refused` / `SocketException` .
Firebase Admin SDK	External Cloud (Google)	NO (Optional)	`extensions.py` detects absence of `firebase_credentials.json` or network timeout, prints `[INFO] Running in local mode (notifications disabled)` and continues serving all REST and cryptographic requests normally.
Vite Dev Server	Web Process	NO (Admin only)	Mobile clients function 100% without Admin Web; Admin Web is only required for registrar/warden triage.

6. ENVIRONMENT VARIABLES

The backend configuration loader in `backend/config.py` uses standard library reading of `.env` files located in `backend/.env` or repository root.

Variable Name	Purpose	Required?	Example Format (Placeholders)	Used In	Safe to Expose to Client?
`DATABASE_URL`	SQLAlchemy connection string	NO (Has dev fallback)	`mysql+pymysql://<user>:<password>@localhost:3306/attentify`	`config.py:33`	**NO (CRITI SECRET)**
`GATE_PASS_SECRET`	HMAC-SHA256 symmetric signing secret for QR Gate Passes	**YES (Fails fast if missing)**	`campusos_gate_sec_<64_char_hex_or_alphanumeric>`	`config.py:40`, `utils/crypto.py:17`	**NO (CRITI SECRET)**
`PASSPORT_INSTITUTION_PRIVATE_KEY`	Base64-encoded 32-byte Ed25519 private key for signing Passport certificates	**YES (Fails fast if missing)**	`<44_char_base64_string ≥` (32 raw bytes)	`passport_service.py:80`	**NO (TOP SECRET ROOT KEY**
`PASSPORT_INSTITUTION_KEY_ID`	Identifier for active institutional signing key	NO (Defaults to `campusos-root-2026-v1`)	`campusos-root-2026-v1`	`config.py:43`, `passport_service.py:71`	YES (Public identifier)
`JWT_SECRET`	HMAC-SHA256 secret for user session tokens	NO (Has dev fallback, **REQUIRED FOR PROD**)	`campusos_jwt_sec_<64_char_hex>`	`config.py:53`, `utils/crypto.py:77`	**NO (CRITI SECRET)**
`JWT_EXPIRATION_HOURS`	Lifespan of issued Bearer JWTs	NO (Defaults to `168`)	`168` (Integer hours, e.g. 7 days)	`config.py:54`, `utils/crypto.py:102`	YES (Configura parameter
`FLASK_DEBUG`	Controls Flask debug and reload mode	NO (Defaults to `false`)	`false` or `true`	`app.py:162`, `run.py:12`	YES (Set `false` fo multi-clien testing)
`PORT`	Listening TCP port for backend	NO (Defaults to `5000`)	`5000`	`run.py:11`	YES (Port number)
`VITE_API_BASE_URL`	Admin Web API target URL	NO (Defaults to `/api`)	`http://192.168.1.100:5000`	`admin-web/src/services/api/api-client.ts:22`	YES (Client side URL)

Environment Files Matrix

`backend/.env` **(LAN Testing Configuration)**

```
# CampusOS LAN Deployment Environment Configuration
DATABASE_URL=mysql+pymysql://root:nandasourav766@localhost:3306/attentify
GATE_PASS_SECRET=campusos_gate_sec_7a9f82d1c4b8e053a6f1d2e4c8b7a9f0
PASSPORT_INSTITUTION_KEY_ID=campusos-root-2026-v1
PASSPORT_INSTITUTION_PRIVATE_KEY=t6s2qBfceKH/2cFn7Fy/yD7NdY+650e369ujP4/VD2E=
JWT_SECRET=campusos_jwt_sec_8f9c2d1b4e053a6f1d2e4c8b7a9f0e3d
JWT_EXPIRATION_HOURS=168
FLASK_DEBUG=false
PORT=5000
```

`backend/.env.production` **(Production Hardened Template)**

```
# CampusOS Production Environment Configuration
DATABASE_URL=mysql+pymysql://campusos_user:<GENERATE_STRONG_DB_PASSWORD>@127.0.0.1:3306/attentify
GATE_PASS_SECRET=<GENERATE_64_CHAR_HEX_SECRET>
PASSPORT_INSTITUTION_KEY_ID=campusos-root-2026-v1
PASSPORT_INSTITUTION_PRIVATE_KEY=<GENERATE_BASE64_ED25519_PRIVATE_KEY>
JWT_SECRET=<GENERATE_64_CHAR_HEX_SECRET>
JWT_EXPIRATION_HOURS=24
FLASK_DEBUG=false
PORT=5000
```

7. DATABASE SETUP

Database Initialization Mechanism

The repository does **not** use Alembic or Flask-Migrate directories. Instead, it utilizes an **idempotent, non-destructive additive programmatic migration engine** implemented in `backend/models/schema.py`:
1. `db.create_all()` creates all 27 SQLAlchemy tables if they do not exist.
2. An additive inspection phase inspects MySQL `information_schema.COLUMNS` and `information_schema.TABLES` to alter and add missing columns (`notices`, `notification_deliveries`, `device_tokens`, `mess_menus`, etc.).
3. An indexing phase validates and creates composite performance indexes across high-traffic transit tables (`idx_gp_student_status`, `idx_me_student_date_meal`, etc.).
4. This mechanism runs automatically every time `create_app()` executes.

Step-by-Step MySQL Setup Commands

1. Log in to MySQL as root:

```
mysql -u root -p
```

2. Execute Database and User Provisioning Script:

```
-- 1. Create the official CampusOS database with UTF-8 support
CREATE DATABASE IF NOT EXISTS attentify
CHARACTER SET utf8mb4
COLLATE utf8mb4_unicode_ci;

-- 2. Verify creation
SHOW DATABASES LIKE 'attentify';

-- 3. (Optional for production) Create dedicated least-privilege user
CREATE USER IF NOT EXISTS 'campusos_user'@'localhost' IDENTIFIED BY 'CampusOS@Secure2026';
GRANT ALL PRIVILEGES ON attentify.* TO 'campusos_user'@'localhost';
FLUSH PRIVILEGES;

-- 4. Exit MySQL prompt
EXIT;
```

3. Initialize Schema:

Run Python interactive app context to verify instant table creation:

```
python -c "from app import app, db; app.app_context().push(); print('Tables:', len(db.metadata.tables))"
```

Expected Output: `Tables: 27`

---

8. SECRET SETUP

1. Gate Pass Signing Secret (`GATE_PASS_SECRET`)

- **Primitive:** HMAC-SHA256 symmetric signature.
- **Generation Command (PowerShell / Linux):**
  `bash python -c "import secrets; print('campusos_gate_sec_' + secrets.token_hex(32))"`
- **Loss Impact:** If rotated while students have unexpired gate passes, existing passes will fail verification at the gate with `"Cryptographic verification failed: Signature mismatch / tampering detected"`.
- **Mitigation:** Only rotate during scheduled maintenance windows when all student gate passes have reached `RETURNED` or `EXPIRED` status.

2. Passport Institutional Signing Key (`PASSPORT_INSTITUTION_PRIVATE_KEY`)

- **Primitive:** Ed25519 (Edwards-curve Digital Signature Algorithm, RFC 8032).
- **Generation Command:**
  `bash python -c "import base64; from cryptography.hazmat.primitives.asymmetric import ed25519; k = ed25519.Ed25519PrivateKey.generate(); print(base64.b64encode(k.private_bytes_raw()).decode())"`
- **Loss Impact:** If the private key is lost or modified, offline verifiers and the mess verification engine will reject all previously issued student device certificates (`DeviceCertificate`).
- **Mitigation:** The active institution private key must be safely backed up in cold storage.

3. JWT Session Signing Secret (`JWT_SECRET`)

- **Primitive:** HMAC-SHA256 compact token signing.
- **Generation Command:**
  `bash python -c "import secrets; print('campusos_jwt_sec_' + secrets.token_hex(32))"`
- **Loss Impact:** Invalidates all active user sessions; all mobile clients and admin portals will receive `401 Unauthorized` and will be required to log in again.

---

9. BACKEND STARTUP

Development / LAN Testing Server

The development entry point is configured in `backend/run.py` or `backend/app.py`:
- Listens on `0.0.0.0` (all network interfaces).
- Port: `5000`.
- Multi-threading enabled by default in Flask 2.0+.

```
cd backend
python app.py
```

Expected Startup Output:

```
[OK] [PASSPORT] Institutional signing key configured: keyId=campusos-root-2026-v1, pubKey=8zW...
[OK] Firebase Admin initialized successfully.
 * Serving Flask app 'app'
 * Debug mode: off
WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
 * Running on all addresses (0.0.0.0)
 * Running on http://127.0.0.1:5000
 * Running on http://192.168.1.100:5000
Press CTRL+C to quit
```

Production WSGI Server (Waitress on Windows / Gunicorn on Linux)

Windows Server (via Waitress):

```
pip install waitress
python -c "from waitress import serve; from app import app; print('Serving on 0.0.0.0:5000...'); serve(app, host='0.0.0.0', port=5000, threads=8)"
```

Linux Server (via Gunicorn):

```
pip install gunicorn
gunicorn --bind 0.0.0.0:5000 --workers 4 --threads 2 --timeout 60 app:app
```

---

10. FIREWALL / NETWORK CONFIGURATION

The backend host must permit incoming TCP traffic on port `5000` (Flask API) and port `3000` (Admin Web) across the local subnet.

Windows Firewall Configuration (Administrator PowerShell):

```
# Open inbound TCP port 5000 for Flask REST API
New-NetFirewallRule -DisplayName "CampusOS Backend REST API (5000)" `
    -Direction Inbound -LocalPort 5000 -Protocol TCP -Action Allow -Profile Private,Domain

# Open inbound TCP port 3000 for Admin Web Control Plane
New-NetFirewallRule -DisplayName "CampusOS Admin Web (3000)" `
    -Direction Inbound -LocalPort 3000 -Protocol TCP -Action Allow -Profile Private,Domain

# Verify rules are active
Get-NetFirewallRule -DisplayName "CampusOS*" | Select-Object DisplayName, Enabled, Direction, Action
```

Linux Firewall Configuration (UFW):

```
sudo ufw allow 5000/tcp comment "CampusOS Flask API"
sudo ufw allow 3000/tcp comment "CampusOS Admin Web"
sudo ufw status verbose
```

Finding the Server LAN IP:

- **Windows:** Run `ipconfig` and look for `IPv4 Address` under your active Wi-Fi or Ethernet adapter (e.g., `192.168.1.100`).
- **Linux:** Run `ip -br a` or `hostname -I`.

---

11. FLUTTER CONNECTION

Actual Configuration Location

In the Flutter codebase, the API base URL is defined in:
flutter_frontent/lib/core/config/app_config.dart

```
static const String _defaultUrl = 'http://127.0.0.1:5000';
static const String baseUrl =
    String.fromEnvironment('API_BASE_URL', defaultValue: _defaultUrl);
```

Configuring Mobile Devices for Physical Testing

Option A: Compile-Time / Run-Time Parameter (Recommended - Zero Code Changes)

Pass `--dart-define` with the server's actual LAN IP:

```
flutter run --dart-define=API_BASE_URL=http://192.168.1.100:5000
```

Or build the APK for distribution across test phones:

```
flutter build apk --debug --dart-define=API_BASE_URL=http://192.168.1.100:5000
```

Option B: Direct File Configuration

Modify line 10 in `flutter_frontent/lib/core/config/app_config.dart`:

```
static const String _defaultUrl = 'http://192.168.1.100:5000';
```

Android Cleartext Policy Verification

In `flutter_frontent/android/app/src/main/AndroidManifest.xml` line 19:

```
<application
    android:label="CampusOS"
    android:usesCleartextTraffic="true">
```

Verification: `android:usesCleartextTraffic="true"` is **already enabled**. Android devices will connect directly to `http://<SERVER_LAN_IP>:5000` over Wi-Fi without SSL/TLS blocking.

---

12. ADMIN WEB CONNECTION

Configuration Location

In `admin-web/src/services/api/api-client.ts` line 22:

```
const BASE_URL = import.meta.env.VITE_API_BASE_URL || "/api"
```

LAN Deployment Options

Option A: Running Vite on the Server Machine (Recommended)

`admin-web/vite.config.ts` includes a built-in proxy:

```
server: {
  port: 3000,
  host: '0.0.0.0', // Expose to LAN
  proxy: {
    '/api': {
      target: 'http://127.0.0.1:5000',
      changeOrigin: true,
      rewrite: (path) => path.replace(/^\/api/, ''),
    },
  },
}
```

Run from `admin-web/`:

```
npm run dev -- --host 0.0.0.0
```

Any browser on the LAN can navigate to `http://192.168.1.100:3000`. All `/api/*` calls are transparently proxied to Flask on port 5000 with zero CORS friction.

Option B: Standalone Production Build

Create `admin-web/.env.production`:

```
VITE_API_BASE_URL=http://192.168.1.100:5000
```

Build static assets:

```
npm run build
npm run preview -- --host 0.0.0.0 --port 3000
```

---

13. TEST DATA SYSTEM

The repository contains an idempotent, deterministic seed system in:
`backend/seed_test_data.py`

What It Generates

1. **6 Primary Role Users:** Creates or updates accounts with verified emails and roles.
2. **Relational Entity Purge:** Safely purges only the relational child records belonging to the test accounts (leaves, passes, attendances, sessions, meal entries, certificates) without touching other user data.
3. **Hostel Allocation & Room Assets:** Assigns student Aarav Sharma to Room `B-204` with study desk asset `ABH-B204-DSK-01`.
4. **Academic Sessions & Attendance:** Creates active B.Tech CSE Year 3 class `CS301` with lecture session and marked attendance.
5. **Coupons:** Provisions an active campus cafeteria reward coupon.
6. **Student Leave Request & Approved Gate Pass:** Generates an approved leave request and an active `GatePass` with an authentic HMAC-SHA256 signed compact token.
7. **Institutional Notices:** Seeds academic and administrative notices with target deliveries and read tracking.
8. **Mess Menus & Dining Feedback:** Seeds today's Breakfast, Lunch, Snacks, and Dinner with menu items and rating records.
9. **Cryptographic Passport Identity:** Provisions an Ed25519 `DeviceIdentityKey` and signs a `DeviceCertificate` bound to the student user record.
10. **Academic Updates & Complaints:** Seeds class rescheduling announcements and pending hostel maintenance complaints.

Seeding Execution Commands

```
cd backend

# Execute clean seed (idempotent)
python seed_test_data.py

# Verify seeded entities and relationships
python seed_test_data.py --verify

# Optional full relational purge and fresh re-seed
python seed_test_data.py --reset
```

14. SIX TEST ACCOUNTS (DETERMINISTIC MATRIX)

> ⚠ **WARNING**
> These credentials are strictly for isolated LAN and staging validation. They MUST NOT be deployed to any publicly accessible internet endpoint.

Role	Email	Password	Display Name	Registration / Employee ID	Client Interface
Admin	`admin@campusos.edu`	`Admin@1234`	Institutional Registrar & Dean	`ADM-2026-001`	React Admin Web (`:3000`)
Teacher	`teacher@campusos.edu`	`Teacher@1234`	Prof. Sarah Jenkins	`FAC-CSE-104`	Flutter Mobile App
Student	`student@campusos.edu`	`Student@1234`	Aarav Sharma	`2023CSE042`	Flutter Mobile App
Warden	`warden@campusos.edu`	`Warden@1234`	Dr. Rajesh Sharma (Chief Warden)	`WRD-BH-01`	Flutter Mobile App
Mess Staff	`mess@campusos.edu`	`Mess@1234`	Ramesh Kumar (Mess Manager)	`MSS-NORTH-01`	Flutter Mobile App
Gate Security	`security@campusos.edu`	`Security@1234`	Officer Vikram Singh	`SEC-MAIN-007`	Flutter Mobile App

15. PHYSICAL E2E TEST PLAN (6 CORE WORKFLOWS)

Workflow 1: Leave Request -> Gate Pass Lifecycle

1. **Student Phone:** Navigates to Outing/Leave, applies for leave with reason "Home Visit", exit 18:00, return 21:00.
2. **Warden Phone:** Receives leave request under Pending Leave tab. Reviews and taps "Approve". Backend transitions status to `APPROVED` and generates `GatePass` with signed HMAC token.
3. **Student Phone:** Gate pass appears with live dynamic QR code containing the HMAC-SHA256 compact token.
4. **Security Phone:** Opens Gate Scanner, scans student QR code. Scanner verifies token via `POST /security/gate-pass/verify`. Displays student name, roll number, and approved destination. Guard confirms Exit.
5. **Security Phone:** Later when student returns, guard scans student QR again. Backend marks gate pass `RETURNED`.

Workflow 2: BLE / QR Attendance Verification

1. **Teacher Phone:** Starts attendance session for Class `CS301` with transmit power set to Medium. Backend creates session with unique nonce. Teacher device begins BLE advertisement.
2. **Student Phone:** Opens Attendance tab. Discovers teacher's BLE broadcast or scans session QR code fallback.
3. **Student Phone:** Submits attendance payload signed by student identity key.
4. **Backend:** Verifies session nonce and records student attendance as `PRESENT`. Evaluates loyalty rules and issues coupon reward.

Workflow 3: Mess Dining Passport Verification

1. **Student Phone:** Opens Dining Passport tab. Generates dynamic Ed25519 meal presentation token.
2. **Mess Staff Phone:** Opens Dining Scanner, scans student QR code.
3. **Backend:** Calls `POST /mess/entries/verify`. `PassportService.verify_presentation` validates Ed25519 digital signature and certificate validity. Inserts `meal_entries` record. Staff UI displays green success checkmark with student photo and meal type.

Workflow 4: Complaint Resolution Flow

1. **Student Phone:** Submits hostel maintenance complaint regarding study desk lamp.
2. **Admin Web:** Complaint appears on Admin Dashboard under Active Complaints. Registrar assigns complaint to maintenance contractor and updates status to `IN_PROGRESS`.
3. **Student Phone:** Receives updated status notification. Once fixed, Admin marks complaint `RESOLVED`.

Workflow 5: Targeted Institutional Notice Broadcast

1. **Admin Web:** Composes emergency academic notice targeted to `B.Tech CSE Year 3`. Taps "Publish Notice".
2. **Backend:** Resolves matching student recipients and creates `notification_deliveries` records. Dispatches FCM push notification.
3. **Student Phone:** Receives real-time push notification and banner on Student Home Screen. Opens notice, updating read status to `READ`.

Workflow 6: Visitor Pass Issuance & Security Clearance

1. **Student Phone:** Submits visitor request for parent arrival.
2. **Warden Phone:** Reviews visitor request and grants approval. Backend issues 6-digit visitor pass code.
3. **Security Phone:** Enters visitor pass code at campus security gate. Clearance confirmed and entry timestamp recorded.

---

16. BACKUP AND RESTORE

Automated Backup Script (Windows PowerShell)

Create `backend/backup_db.ps1`:

```
$Date = Get-Date -Format "yyyyMMdd_HHmmss"
$BackupDir = "D:\DEVLOPEMENT\ezzo_7.5\backups"
if (!(Test-Path $BackupDir)) { New-Item -ItemType Directory -Path $BackupDir | Out-Null }
$OutputFile = "$BackupDir\attentify_backup_$Date.sql"

Write-Host "Creating backup of database 'attentify'..." -ForegroundColor Cyan
& "mysqldump.exe" -u root -p attentify --routines --triggers --events > $OutputFile

if ($LASTEXITCODE -eq 0) {
    Write-Host "Backup successfully created: $OutputFile" -ForegroundColor Green
} else {
    Write-Host "Backup failed with exit code $LASTEXITCODE" -ForegroundColor Red
}
```

Automated Backup Script (Linux Bash)

```
#!/bin/bash
DATE=$(date +"%Y%m%d_%H%M%S")
BACKUP_DIR="/var/backups/campusos"
mkdir -p "$BACKUP_DIR"
OUTPUT_FILE="$BACKUP_DIR/attentify_backup_$DATE.sql"

echo "Creating MySQL backup..."
mysqldump -u root -p attentify --routines --triggers --events > "$OUTPUT_FILE"

if [ $? -eq 0 ]; then
    echo "Backup created successfully at $OUTPUT_FILE"
else
    echo "Backup failed!" >&2
fi
```

Restoration Commands

To restore a snapshot into a clean database:

```
# 1. Re-create clean database
mysql -u root -p -e "DROP DATABASE IF EXISTS attentify; CREATE DATABASE attentify CHARACTER SET utf8mb4 COLLATE utf8mb4_unicode_ci;"

# 2. Import SQL dump
mysql -u root -p attentify < /path/to/attentify_backup_YYYYMMDD_HHMMSS.sql

# 3. Verify table counts
mysql -u root -p -e "SELECT COUNT(*) AS total_tables FROM information_schema.tables WHERE table_schema='attentify';"
```

---

17. LAN DEPLOYMENT RUNBOOK (STEP-BY-STEP EXECUTION)

STEP 1 — Verify Environment Prerequisites

- **Purpose:** Ensure Python, Node, MySQL, and Flutter SDKs are present.
- **Commands:**
  `bash python --version node --version mysql --version`
- **Expected Output:** Python 3.10+, Node v20+, MySQL 8.0+.
- **Common Error:** Command not found.

- **Fix:** Add installation folders to system `PATH`.

STEP 2 — Setup Python Virtual Environment

- **Purpose:** Isolate project dependencies.
- **Commands (Windows):**
  `powershell cd d:\DEVLOPEMENT\ezzo_7.5\backend python -m venv .venv .venv\Scripts\Activate.ps1 pip install --upgrade pip pip install -r requirements.txt`
- **Expected Output:** `Successfully installed Flask Flask-Cors Flask-SQLAlchemy PyMySQL cryptography PyJWT ...`

STEP 3 — Verify and Configure MySQL Database

- **Purpose:** Provision `attentify` database.
- **Commands:**
  `bash mysql -u root -p -e "CREATE DATABASE IF NOT EXISTS attentify CHARACTER SET utf8mb4 COLLATE utf8mb4_unicode_ci;"`
- **Expected Output:** Query OK, 1 row affected.

STEP 4 — Configure Backend Environment File

- **Purpose:** Configure signing secrets and database URI in `backend/.env`.
- **Action:** Ensure `backend/.env` exists and contains:
  `env DATABASE_URL=mysql+pymysql://root:nandasourav766@localhost:3306/attentify GATE_PASS_SECRET=campusos_gate_sec_7a9f82d1c4b8e053a6f1d2e4c8b7a9f0 PASSPORT_INSTITUTION_KEY_ID=campusos-root-2026-v1 PASSPORT_INSTITUTION_PRIVATE_KEY=t6s2qBfceKH/2cFn7Fy/yD7NdY+650e369ujP4/VD2E= JWT_SECRET=campusos_jwt_sec_8f9c2d1b4e053a6f1d2e4c8b7a9f0e3d FLASK_DEBUG=false PORT=5000`

STEP 5 — Run Automated Backend Validation Suites

- **Purpose:** Confirm all cryptographic and domain units pass before launching.
- **Commands:**
  `bash python run_all_backend_tests.py`
- **Expected Output:**
  `Total Suites: 19 | Passed: 19 | Failed: 0`

STEP 6 — Seed Deterministic Multi-Role Data

- **Purpose:** Populate database with verified accounts across all 6 roles.
- **Commands:**
  `bash python seed_test_data.py --verify`
- **Expected Output:**
  `[OK] All 6 role accounts verified. [OK] System is ready for physical multi-device testing.`

STEP 7 — Launch Flask Backend on LAN Interface

- **Purpose:** Serve REST API to LAN devices.
- **Commands:**
  `bash python app.py`
- **Expected Output:** `Running on http://0.0.0.0:5000`

STEP 8 — Open Firewall Ports

- **Purpose:** Allow incoming mobile traffic through OS firewall.
- **Commands (PowerShell Admin):**
  `powershell New-NetFirewallRule -DisplayName "CampusOS Backend" -Direction Inbound -LocalPort 5000 -Protocol TCP -Action Allow`

STEP 9 — Identify Server LAN IP Address

- **Purpose:** Get target IP for mobile clients.
- **Commands:**
  `powershell Get-NetIPAddress -AddressFamily IPv4 -InterfaceAlias "Wi-Fi*","Ethernet*" | Select-Object IPAddress, InterfaceAlias`
- **Example Value:** `192.168.1.100`

STEP 10 — Launch Admin Web Control Plane

- **Purpose:** Start Registrar / Warden control panel.
- **Commands:**
  `bash cd d:\DEVLOPEMENT\ezzo_7.5\admin-web npm install npm run dev -- --host 0.0.0.0`

- **Expected Output:** `➜ Network: http://192.168.1.100:3000/`

STEP 11 — Compile & Distribute Flutter Mobile Client

- **Purpose:** Install CampusOS mobile application on physical Android devices.
- **Commands:**
  `bash cd d:\DEVLOPEMENT\ezzo_7.5\flutter_frontent flutter build apk --debug --dart-define=API_BASE_URL=http://192.168.1.100:5000`
- **Output Artifact:** `build/app/outputs/flutter-apk/app-debug.apk`
- **Installation:**
  `bash adb -s <DEVICE_SERIAL> install -r build/app/outputs/flutter-apk/app-debug.apk`

STEP 12 — Execute 6-Role Device Login Verification

- **Purpose:** Verify authentication from each physical device.
- **Action:** Log in with:
  1. Phone 1: `student@campusos.edu` / `Student@1234`
  2. Phone 2: `teacher@campusos.edu` / `Teacher@1234`
  3. Phone 3: `warden@campusos.edu` / `Warden@1234`
  4. Phone 4: `security@campusos.edu` / `Security@1234`
  5. Phone 5: `mess@campusos.edu` / `Mess@1234`
  6. Laptop Browser: `admin@campusos.edu` / `Admin@1234` at `http://192.168.1.100:3000`

---

18. INTERNET DEPLOYMENT RUNBOOK (CONTROLLED / PRODUCTION)

```
[ Internet Clients (Mobile / Web) ]
              | (HTTPS:443 / TLS 1.3)
              ▼
[ Nginx Reverse Proxy / Cloudflare Edge ]
              | (HTTP:5000 - Reverse Proxied over Unix Socket or 127.0.0.1)
              ▼
[ Gunicorn / Waitress WSGI Application Server (4 Workers) ]
              |
[ CampusOS Flask Application Root ]
              |
[ MySQL 8.0 InnoDB Database Engine (127.0.0.1:3306) ]
```

Production Security Checklist Before Public Exposure

1. **Password Hashing:**
   - *Current State:* Plaintext strings (`User.password`).
   - *Mandatory Fix:* Migrate to `bcrypt.hashpw()` or Argon2id in `routes/auth.py` and `models/user.py`.
2. **Strict JWT Enforcement:**
   - *Current State:* Legacy fallback to `X-User-Id` and query parameters exists in `middleware/auth.py`.
   - *Mandatory Fix:* Disable `X-User-Id` resolution in production (`if not app.config['DEBUG']: reject unauthenticated`).
3. **Explicit JWT Secrets:**
   - *Current State:* Default fallback secret in `config.py`.
   - *Mandatory Fix:* Make `JWT_SECRET` mandatory; raise `RuntimeError` if missing from environment.
4. **Nginx Reverse Proxy Configuration:**
   Create `/etc/nginx/sites-available/campusos.conf` :
   ```nginx
   server {
   listen 80;
   server_name campusos.institution.edu;
   return 301 https://$host$request_uri;
   }

server {
listen 443 ssl http2;
server_name campusos.institution.edu;
```

```
    ssl_certificate /etc/letsencrypt/live/campusos.institution.edu/fullchain.pem;
    ssl_certificate_key /etc/letsencrypt/live/campusos.institution.edu/privkey.pem;
    ssl_protocols TLSv1.2 TLSv1.3;
    ssl_ciphers HIGH:!aNULL:!MD5;

    # Security Headers
    add_header X-Content-Type-Options nosniff;
    add_header X-Frame-Options DENY;
    add_header X-XSS-Protection "1; mode=block";

    # Reverse Proxy to Gunicorn
    location / {
        proxy_pass http://127.0.0.1:5000;
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
        proxy_set_header X-Forwarded-Proto $scheme;
        proxy_connect_timeout 30s;
        proxy_read_timeout 60s;
    }
```

}
```

## 19. DEPLOYMENT BLOCKERS & REMEDIATION MATRIX

| Issue / Vulnerability | Current Implementation | Blocks LAN Testing? | Blocks Internet Production? | Severity | Remediation Action |
|---|---|---|---|---|---|
| **Plaintext Passwords** | `User.password` stored as plain string in MySQL | **NO** (Acceptable for isolated physical lab) | **YES (CRITICAL)** | CRITICAL | Migrate to `bcrypt` / `argon2` hashing before exposing to internet. |
| `X-User-Id` **Header Fallback** | `middleware/auth.py` accepts client-supplied header if no Bearer token | **NO** (Permitted for rapid role switching/testing) | **YES (CRITICAL)** | CRITICAL | Strip `X-User-Id` fallback; strictly enforce `verify_access_token()`. |
| **Hardcoded JWT Secret Fallback** | `config.py` provides static default fallback | **NO** (Working for local runs) | **YES (HIGH)** | HIGH | Require `JWT_SECRET` in environment; fail fast if missing. |
| **HTTP Cleartext Traffic** | Mobile communicates over unencrypted HTTP | **NO** ( `usesCleartextTraffic="true"` configured) | **YES (HIGH)** | HIGH | Deploy TLS cert via Nginx / Cloudflare; re-compile Flutter with HTTPS. |
| **Wildcard CORS** | `CORS(app)` in `app.py` allows all origins `*` | **NO** (Ideal for multi-machine local test) | **YES (MEDIUM)** | MEDIUM | Restrict CORS to explicit institutional domains in `CORS(app, origins=[...])`. |
| **Missing Health Endpoint** | No `/health` route exists in routes or `app.py` | **NO** (Not needed for basic operation) | NO | LOW | *REQUIRES DECISION:* Implement `GET /health` if load balancers require uptime probing. |
| **Local Logging via** `print()` | Output printed to standard console | **NO** (Readable in dev console) | NO | MEDIUM | Configure standard `logging` with timed rotating file handlers. |
| **Missing FCM Service Key** | Handled gracefully in `extensions.py` | **NO** (Notifications gracefully disable) | NO (Non-push features work) | LOW | Provide `firebase_credentials.json` if physical push is required. |

## 20. EXACT COPY-PASTE COMMANDS

### Windows Server (Command Line / PowerShell)

```
1. Initialize environment
cd d:\DEVLOPEMENT\ezzo_7.5\backend
python -m venv .venv
.venv\Scripts\Activate.ps1
pip install -r requirements.txt

2. Setup MySQL database
mysql -u root -p -e "CREATE DATABASE IF NOT EXISTS attentify CHARACTER SET utf8mb4 COLLATE utf8mb4_unicode_ci;"

3. Seed deterministic physical test records
python seed_test_data.py --verify

4. Open Windows Firewall ports
New-NetFirewallRule -DisplayName "CampusOS-5000" -Direction Inbound -LocalPort 5000 -Protocol TCP -Action Allow -ErrorAction SilentlyContinue

5. Start Flask backend
python app.py
```
```

Linux Server (Bash)

```
# 1. Initialize environment
cd /opt/campusos/backend
python3 -m venv .venv
source .venv/bin/activate
pip install -r requirements.txt

# 2. Setup MySQL database
sudo mysql -e "CREATE DATABASE IF NOT EXISTS attentify CHARACTER SET utf8mb4 COLLATE utf8mb4_unicode_ci;"

# 3. Seed test data
python seed_test_data.py --verify

# 4. Open firewall
sudo ufw allow 5000/tcp
sudo ufw allow 3000/tcp

# 5. Start production WSGI
gunicorn --bind 0.0.0.0:5000 --workers 4 app:app
```

Flutter Client Compilation (Deploy to 5 Phones)

```
cd d:\DEVLOPEMENT\ezzo_7.5\flutter_frontent
# Replace 192.168.1.100 with your actual server LAN IP
flutter build apk --debug --dart-define=API_BASE_URL=http://192.168.1.100:5000
```

React Admin Web Launch

```
cd d:\DEVLOPEMENT\ezzo_7.5\admin-web
npm install
npm run dev -- --host 0.0.0.0
```

---

21. TROUBLESHOOTING RUNBOOK

Symptom / Error Message	Root Cause	Exact Resolution
`CRITICAL CONFIGURATION ERROR: 'GATE_PASS_SECRET' environment variable is not configured`	Missing `GATE_PASS_SECRET` in `backend/.env`	Add `GATE_PASS_SECRET=campusos_gate_sec_7a9f82d1c4b8e053a6f1d2e4c8b7a9f0` to `backend/.env`.
`Institutional signing key is not configured. Set the 'PASSPORT_INSTITUTION_PRIVATE_KEY' environment variable`	Missing Ed25519 root private key	Add `PASSPORT_INSTITUTION_PRIVATE_KEY=t6s2qBfceKH/2cFn7Fy/yD7NdY+650e369ujP4/VD2E=` to `backend/.env`.
`PyMySQL.err.OperationalError: (2003, "Can't connect to MySQL server")`	MySQL service is stopped or port 3306 blocked	Start MySQL service: `net start MySQL80` (Windows) or `sudo systemctl start mysql` (Linux).
`SocketException: OS Error: Connection refused, errno = 111` on mobile	Phone cannot reach backend port 5000	1. Ensure phone and server are on identical Wi-Fi SSID. 2. Confirm Wi-Fi AP has "Client Isolation" disabled. 3. Open port 5000 in Windows Firewall.
`Login failed: This account is locked to another device` (Student login)	Student `device_id` does not match database record	Execute reset command: `python -c "from app import app, db; from models.user import User; app.app_context().push(); u = User.query.filter_by(email='student@campusos.edu').first(); u.device_id = None; db.session.commit(); print('Device unlocked')"`
`Cryptographic verification failed: Signature mismatch / tampering detected` at Security Gate	`GATE_PASS_SECRET` on server was rotated after pass was issued	Re-issue gate pass from Student phone or re-run `python seed_test_data.py`.
`Cryptographic presentation rejected: Verification failed` at Mess Scanner	Student device identity certificate signed by different institution key ID	Ensure student device certificate was generated against current `PASSPORT_INSTITUTION_PRIVATE_KEY`.

---

22. FINAL DEPLOYMENT CHECKLIST

Before beginning the physical testing session, verify off all 10 checkpoints:

- [ ] **Database Running:** MySQL server is active and accessible via `DATABASE_URL`.
- [ ] **Table Count Verified:** Exactly 27 relational tables present in database `attentify`.
- [ ] **Deterministic Seed Loaded:** `python seed_test_data.py --verify` exits cleanly with `0` errors.
- [ ] **Backend Secrets Set:** `GATE_PASS_SECRET` and `PASSPORT_INSTITUTION_PRIVATE_KEY` populated in `.env`.
- [ ] **Backend Bound to LAN:** Flask listening on `0.0.0.0:5000`.
- [ ] **Firewall Ports Open:** Inbound TCP 5000 and 3000 allowed on server.
- [ ] **Wi-Fi Isolation Disabled:** Server and mobile phones can ping each other across LAN.

- [ ] **Flutter APK Built:** Compiled with `--dart-define=API_BASE_URL=http://<SERVER_LAN_IP>:5000`.
- [ ] **Cleartext Traffic Enabled:** `android:usesCleartextTraffic="true"` confirmed in `AndroidManifest.xml`.
- [ ] **Admin Web Accessible:** Vite dev server accessible from client browsers at `http://<SERVER_LAN_IP>:3000`.

---

23. ROLLBACK PROCEDURE

If the backend encounters unexpected runtime data corruption during multi-device testing:

```
# 1. Stop Flask backend (CTRL+C)

# 2. Execute automated deterministic database purge & re-seed
cd d:\DEVLOPEMENT\ezzo_7.5\backend
python seed_test_data.py --reset

# 3. Alternatively, restore previous clean MySQL snapshot
mysql -u root -p attentify < ..\backups\attentify_clean_predeploy.sql

# 4. Clear client-side offline caches on test mobile devices
# (In Android Settings → Apps → CampusOS → Storage → Clear Storage & Cache)

# 5. Restart Flask backend
python app.py
```